NEMOX AI RICO-7

COMPLETE TECHNICAL ARCHITECTURE | SYNTX FORM

Engineering handover based on inspected source code · NEMOX_AI_RICO7_COMPLETE_SOFTWARE.zip

ARCHITECTURE STATUS: IMPLEMENTED BEHAVIOUR DISTINGUISHED FROM DERIVED VIEWS AND PROPOSED EXTENSIONS | DATE: 1 NOVEMBER 2026

CONFIDENTIAL | PROPRIETARY | NO USE WITHOUT WRITTEN APPROVAL

This document and the architecture, SYNTX representation, reconstruction and diagrams it contains are the confidential work of Ottavio Braun, SYNTX SYSTEM. No right of use is granted. Any use, reproduction, disclosure, adaptation, implementation or AI training — in whole or in part — requires the prior, explicit, written approval of Ottavio Braun for a named purpose. Receipt or access creates no licence. Full terms: final page.

NUTZUNG, WEITERGABE ODER UMSETZUNG – GANZ ODER TEILWEISE – NUR NACH AUSDRÜCKLICHER SCHRIFTLICHER FREIGABE DURCH OTTAVIO BRAUN.

STATUS SEMIOTICS ● IMPLEMENTED ◐ DERIVED ◌ CONCEPTUAL ⊕ PROPOSED EXTENSION ◆ GAP / BOUNDARY

EDGES → CALL / DATA FLOW ⇢ LOGICAL LINK ONLY ⇢ INVERSE TRACE ⇢ MISSING LINK

01 BASELINE, SCOPE & RECONSTRUCTION METHOD §1

- Existing unified nemox-developer-tools project as baseline. The original 17-page PDF is retained as historical design reference — not as evidence that every design feature exists in runtime.

FORWARD RECONSTRUCTION

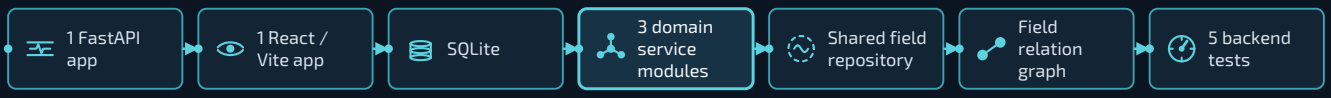


INVERSE RECONSTRUCTION



The installed SYNTX Recursive Deep Sweep and Inverse Sweep were applied as relationship / dependency tracing procedures; their proprietary internal material is not reproduced.

ACTUAL IMPLEMENTATION



◆ IMPLEMENTATION BOUNDARY

Deterministic regex / rule-based demonstration. No external LLM and no nemox systems are connected.

01 SYSTEM CONTEXT



© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

02 COMPLETE SOURCE-CODE INVENTORY

§2 · FILE · LAYER · FUNCTIONS · LINES

◆ Every file of the inspected package, grouped by architectural layer. Layer labels as in the source: API / orchestration, Shared Field Core, Solution A / B / C, Automated tests, Runtime, Frontend, Documentation / config.

DOCUMENTATION / CONFIG

README.md	configuration / UI	47
backend/requirements.txt	configuration / UI	5

API / ORCHESTRATION

backend/api/__init__.py	configuration / UI	0
backend/api/main.py	Request, Decision, Correction, Confirmation, Subsequent, Answers, invalid, health, intent, decision, feedback, confirmation, next_decision, decision_history, gaps, gap_resolution, graph, reset	47

SHARED FIELD CORE

backend/fieldcore/__init__.py	configuration / UI	0
backend/fieldcore/core.py	now, uid, connection, init, FieldRepository, FieldContextResolver, create, get, transition, relation, graph, reset, resolve	88

SOLUTION A · INTENT & CONTEXT INSPECTOR

backend/intent_inspector/__init__.py	configuration / UI	0
backend/intent_inspector/service.py	inspect	19

SOLUTION B · RECURSIVE FEEDBACK & DECISION TRACE

backend/feedback_engine/__init__.py	configuration / UI	0
backend/feedback_engine/service.py	decide, correct, confirm, retrieve, subsequent, history	49

SOLUTION C · UNDERSTANDING GAP DETECTOR

backend/gap_detector/__init__.py	configuration / UI	0
backend/gap_detector/service.py	capability, analyze, evaluate, resolve	24

AUTOMATED TESTS

backend/tests/test_system.py	clean, test_intent_trace, test_feedback_scope_and_confirm, test_gap_resolution, test_state_validation, test_relation_scope	36

RUNTIME

backend/Dockerfile	configuration / UI	8
docker-compose.yml	configuration / UI	10
start.sh	configuration / UI	17

FRONTEND

frontend/Dockerfile	configuration / UI	6
frontend/index.html	configuration / UI	1
frontend/package.json	configuration / UI	1
frontend/src/components/FieldGraph.tsx	configuration / UI	8
frontend/src/main.tsx	configuration / UI	13
frontend/src/services/api.ts	configuration / UI	1
frontend/src/style.css	configuration / UI	1
frontend/tsconfig.json	configuration / UI	1
frontend/vite.config.ts	configuration / UI	2

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

LEGACY · LEGACY/02_EXECUTABLE_SOURCE/NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/

01_user_intent_inspector/Dockerfile	Solution A · configuration / UI	8
02_recursive_feedback_engine/Dockerfile	Solution B · configuration / UI	8
03_understanding_gap_detector/Dockerfile	Solution C · configuration / UI	8
docker-compose.yml	Runtime · configuration / UI	10

29 FILES LISTED IN THE INVENTORY

SUM OF LISTED LINES · 418

03 DOMAIN LOGIC & BOUNDARIES

§3 · THREE DOMAINS · DIAGRAM 02

◆ Three deterministic logistics demonstrations share one field core. Each domain is read as: situation › implemented rule › failure and boundary behaviour.

A Intent & Context Inspector



SITUATION

A freight-forwarding request may refer to a prior shipment and request a delivery-date or cost change. The developer needs the source request, candidate shipment records, extracted constraints and unresolved references.

IMPLEMENTED PROCESSING RULE

inspect() uses case-insensitive regex for earlier / cheaper; a historical cue selects both synthetic HISTORY records; if candidates != 1, historical_shipment_reference remains unresolved. Four fields and three relations are created.

FAILURE & BOUNDARY BEHAVIOUR

Unknown expressions produce empty requirements. Candidate selection does not resolve a unique historical shipment. No actual date arithmetic or booking action.

B Recursive Feedback & Decision Trace



SITUATION

A developer records a carrier decision, receives a correction, confirms / rejects it and observes subsequent decision selection for a matching operation and route.

IMPLEMENTED PROCESSING RULE

decide() inserts a snapshot; correct() validates decision scope and inserts PROPOSED; confirm() updates status and creates a confirmed Feedback field; retrieve() filters CONFIRMED feedback by scope, operation and route; subsequent() chooses latest matching preferred_action.

FAILURE & BOUNDARY BEHAVIOUR

Feedback matching uses no shipment ID, carrier constraints or temporal expiry. Confirmed Feedback field has no relation to Decision field. confirm() uses two transactions.

C Understanding Gap Detector



SITUATION

A logistics instruction must supply capability-required attributes before an operation can be considered complete.

IMPLEMENTED PROCESSING RULE

capability() picks cheaper_shipping if text contains cheaper / lower cost, otherwise booking. evaluate() finds required missing attributes and frontier nodes whose direct dependencies are present. resolve() merges nonblank answers and re-evaluates.

FAILURE & BOUNDARY BEHAVIOUR

Unknown capability defaults to booking. No graph cycle detection or minimal global clarification set. Gap record references a Request field by logical ID without DB FK.

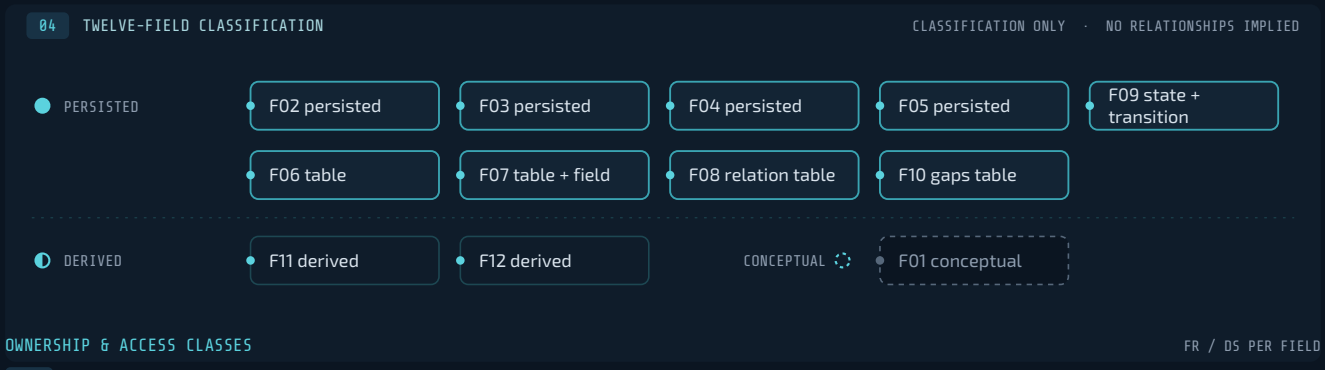
CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

04 TWELVE-FIELD ARCHITECTURE | ACTUAL REPRESENTATIONS

§4 · F01 – F12 · DIAGRAM 04

◆ Twelve canonical fields, each mapped to what actually exists in code: persisted table or FieldRecord, derived projection, or concept only.

Common semantic responsibility (identical for all twelve): the field supports developer understanding of user requests, operational decisions or longitudinal changes. Common lifecycle rule: source event and lifecycle are recorded only where code explicitly writes a FieldRecord or transition. No general event sourcing or field payload version history exists.



OWNERSHIP & ACCESS CLASSES — FR / DS PER FIELD

FR FieldRepository.create / get / transition / relation / graph + FieldContextResolver.resolve

DS Domain service and its corresponding SQLite table or derived API result

F01 User — DS

Conceptual; scope string is only a proxy; no User table

`scope: str; no identity, organization or role columns`

F02 Request — FR

Persisted fields.type=Request, payload.text

`fields.id UUID text, type Request, scope text, state text, version int, payload JSON {text:str}, source_refs JSON, created ISO UTC`

F03 Operational Context — FR

Persisted OperationalContext in A; context JSON in decisions

`fields.payload {candidates:list, selected:null} in A; decisions.context JSON in B`

F04 Intent — FR

Persisted fields.type=Intent in A

`fields.payload {goal:str, unresolved:list[str]}`

F05 Requirement — FR

Persisted fields.type=Requirement in A; capability registry in C

`fields.payload {items:list[{name,value,evidence}]} in A; CAPABILITIES required / questions / dependencies in C`

F06 Agent Decision — DS

Persisted decisions table; not a FieldRecord

`decisions {id,scope,context JSON,action,created}; subsequent() returns baseline, result, retrieved, changed`

F07 Feedback — DS

Persisted feedback table and confirmed FieldRecord

`feedback {id,decision_id,scope,correction JSON,status,created}; confirmed FieldRecord.payload {decision_id,correction,feedback_id}`

F08 Relation — FR

Persisted relations table; no universal semantic ontology

`relations {id,source,target,kind,scope,valid,source_event,created}; both endpoints fields FK`

F09 State — FR

fields.state + transitions table; feedback.status separately

`fields.state INITIAL / ENRICHED / UNRESOLVED / CORRECTED / CONFIRMED / SUPERSEDED; transitions old_state / new_state / event / version`

F10 Understanding Gap — DS

Persisted gaps table; computed frontier, no Gap FieldRecord

`gaps {id,request_id,capability,scope,values_json,created}; evaluate returns missing / frontier / questions / complete / edges`

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

F11 Developer Inspection — FR

Derived API JSON + frontend graph; not persisted

`graph response {fields,relations,transitions}; UI JSON result and ReactFlow nodes / edges`

F12 Longitudinal Context — D5

Derived confirmed feedback retrieval; no dedicated field

`retrieve(scope,operation,route) derives confirmed correction list; no dedicated history field`

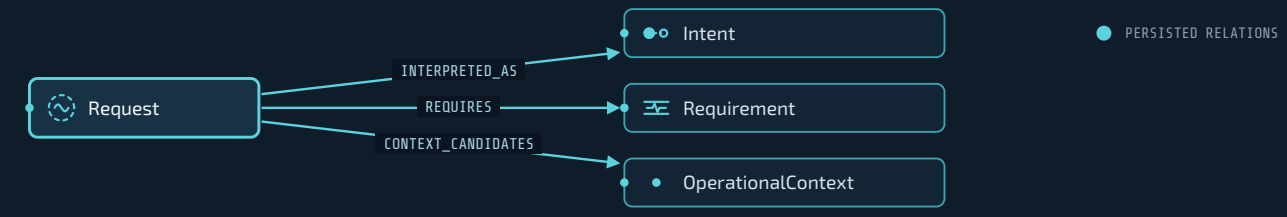
05 FIELD COUPLING & STATE MACHINE

§5 · DIAGRAMS 05 · 06 · 07 · 08

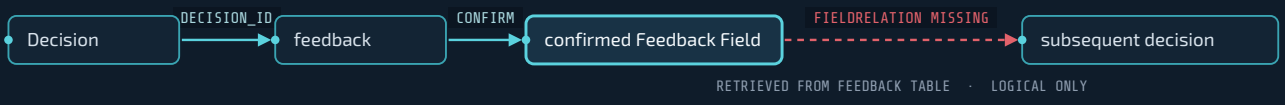
- How fields are coupled by triggers, which effect each coupling actually has in code, and which couplings exist only logically.

FROM › TO	TRIGGER / PRECONDITION	ACTUAL EFFECT / SOURCE
F02 › F04	`POST /api/intent; valid text`	INTERPRETED_AS; intent_inspector.inspect
F02 › F05	`POST /api/intent`	REQUIRES; intent_inspector.inspect
F02 › F03	`POST /api/intent`	CONTEXT_CANDIDATES; intent_inspector.inspect
F06 › F07	`POST /api/feedback; decision in scope`	feedback.decision_id FK; feedback_engine.correct
F07 › F09	`POST confirm; status PROPOSED`	CONFIRMED feedback row; Feedback field ENRICHED then CONFIRMED
F07 › F12 › F06	`POST subsequent; scope / operation / route match`	retrieve() and subsequent(); no stored FieldRelation between these objects
F02 › F10	`POST /api/gaps`	gaps.request_id logical link; analyze()
F10 › F03	`POST /api/gaps/{id}/resolve`	values_json update only; no Context FieldRecord created

06 FIELD COUPLING A



07 FIELD COUPLING B



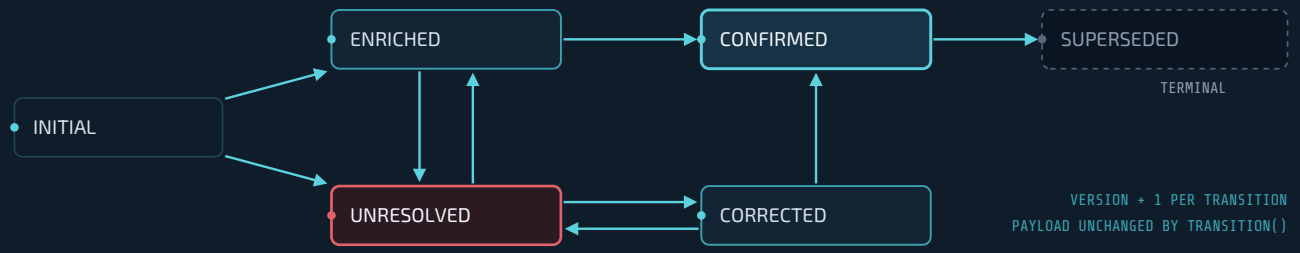
08 FIELD COUPLING C



CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

STATE MACHINE · ALLOWED STATE TRANSITIONS · FIELDCORE/CORE.PY

DIAGRAM 05 · FIELD LIFECYCLE · 9 ALLOWED TRANSITIONS, NO OTHERS

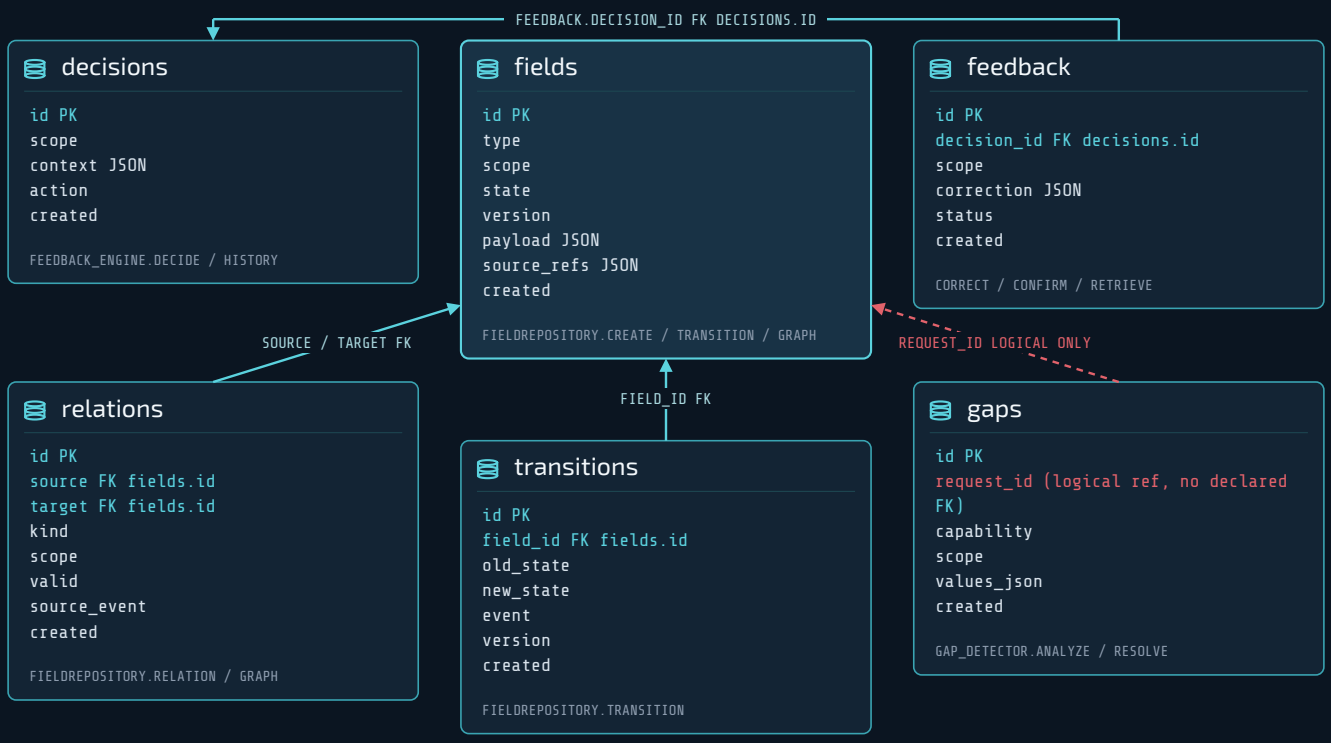


INITIAL › ENRICHED or UNRESOLVED; ENRICHED › UNRESOLVED or CONFIRMED; UNRESOLVED › ENRICHED or CORRECTED; CORRECTED › CONFIRMED or UNRESOLVED; CONFIRMED › SUPERSEDED; SUPERSEDED terminal. Field version increments for each transition, with optimistic check only when the caller supplies expected. Field payload does not change in transition().

05 DATABASE SCHEMA | IMPLEMENTED SQLITE TABLES

§5 · DIAGRAM 11 · DATA RELATIONSHIPS

- Six tables. Foreign keys are declared for relations, transitions and feedback; gaps references its Request field by logical ID only.



◆ PERSISTENCE BOUNDARY

DB path derived from backend/fieldcore/core.py: ROOT/data/nemox.sqlite3. init() runs on api.main import and creates tables with CREATE TABLE IF NOT EXISTS. No migration framework, no per-tenant authentication and no event-envelope table. All writes use sqlite3 connection() with commit / rollback; operations spanning multiple separate connections are not atomic.

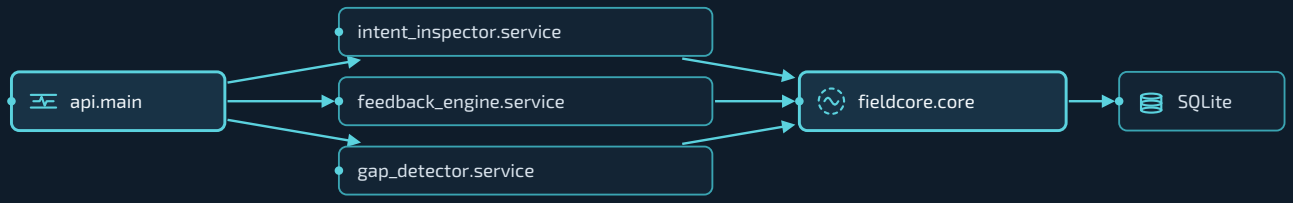
CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

06 BACKEND IMPLEMENTATION ARCHITECTURE

§6 · DIAGRAMS 09 · 03 · 21 · 22

- api.main initialises the SQLite schema, registers CORS localhost:3000, defines Pydantic request types and routes, and maps ValueError to HTTP 400. Each service imports fieldcore directly.

09 BACKEND COMPONENTS



COMPONENT	FUNCTIONS / DATA FLOW / STATE	SOURCE
FieldRepository	create / get / transition / relation / graph / reset; fields / relations / transitions SQLite	backend/fieldcore/core.py
FieldContextResolver	resolve(scope, origin) iterates valid undirected connected relation endpoints; graph output	backend/fieldcore/core.py
Intent	inspect(request, scope) regex + HISTORY, creates 4 fields + 3 relations	backend/intent_inspector/service.py
Feedback	decide, correct, confirm, retrieve, subsequent, history; decisions / feedback	backend/feedback_engine/service.py
Gap	capability, analyze, evaluate, resolve; gaps table	backend/gap_detector/service.py
API	HTTP request validation, routing, ValueError handler	backend/api/main.py

COUPLING BOUNDARY

No service interface abstraction and no dependency-injection container: each service imports fieldcore directly.

03 SHARED FIELD CORE



21 STATE AND EVENT STORAGE



22 CONTEXT GRAPH TRAVERSAL



CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

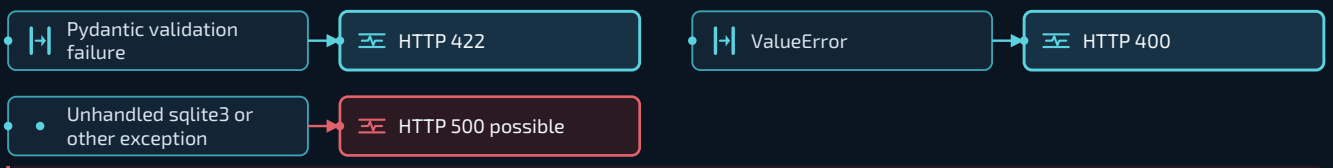
07 COMPLETE IMPLEMENTED API CONTRACTS

57 · 11 ROUTES · DIAGRAM 12

Every route with request shape, response shape and handler. Error semantics and access boundaries follow below.

METHOD	ROUTE	REQUEST	RESPONSE	HANDLER
GET	/api/health	none	{ok:true}	health()
POST	/api/intent	{text:str, scope?:str, values?:object}	request, requirements, candidates, unresolved, field_ids, trace	inspect()
POST	/api/decisions	{scope?:str, action:str, context?:object}	id, scope, action, context	decide()
POST	/api/feedback	{scope?:str, decision_id:str, correction:object}	id, decision_id, scope, correction, status	correct()
POST	/api/feedback/{fid}/confirm	{scope?:str, accept:bool}	id, status, field	confirm()
POST	/api/subsequent	{scope?:str, operation:str, route:str, base_action:str}	baseline, result, retrieved, changed	subsequent()
GET	/api/history/{scope}	scope path	decisions, feedback	history()
POST	/api/gaps	{text:str, scope?:str, values?:object}	id, capability, values, missing, blocking_frontier, questions, complete, edges	analyze()
POST	/api/gaps/{gid}/resolve	{scope?:str, answers:object}	gap evaluation object	resolve()
GET	/api/graph/{scope}	scope path	fields, relations, transitions	FieldRepository.graph()
POST	/api/reset	{}	{reset:true}	FieldRepository.reset()

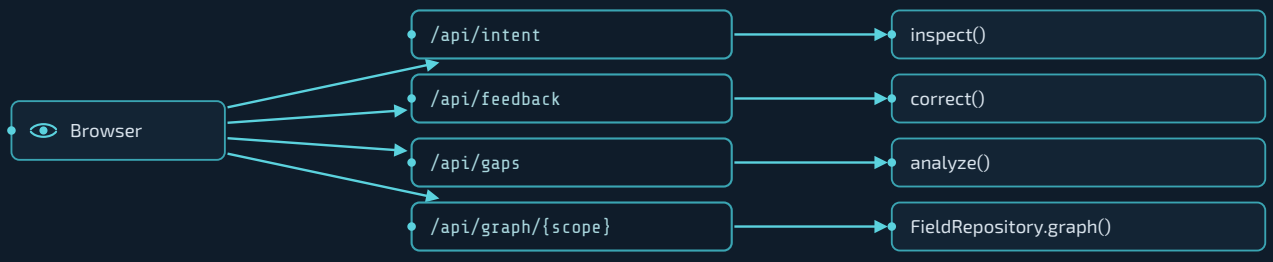
ERROR SEMANTICS



ACCESS BOUNDARY

No authentication, authorization, rate limiting or cross-tenant enforcement on reset. GET history and graph are scope-filtered by URL only.

12 API INTERACTION



CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

08 PROCESSING PATHS & INVERSE TRACES | A

§8 · INTENT & CONTEXT INSPECTOR

FORWARD PROCESSING

10 PROCESSING STEPS

PROCESSING STEP = CALL OR DATA STEP IN THE REQUEST PATH · NOT A FIELDRECORD STATE TRANSITION (STATE MACHINE: P. 06)

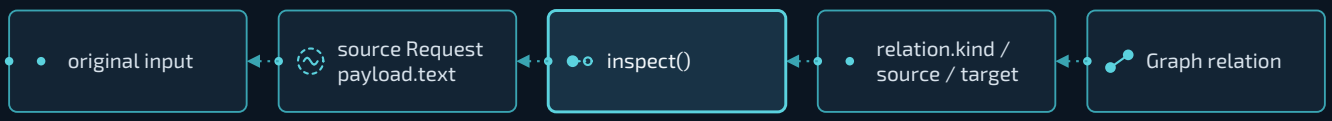


STEP 6 · TRANSITIONS FOR THREE DERIVED FIELDS = THE FIELDRECORD STATE TRANSITIONS WRITTEN BY THIS PATH (P. 06)

INVERSE TRACE

READ RIGHT TO LEFT · RESULT › SOURCE

INVERSE ARROWS = TRACE STEPS, NOT STORED RELATIONS



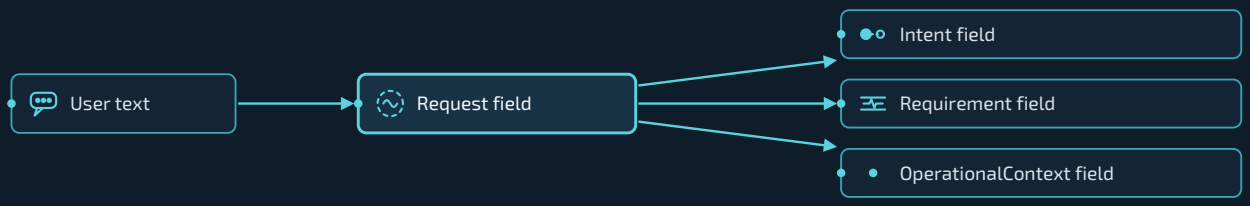
PROVENANCE BOUNDARY

Synthetic HISTORY has no persistent provenance record beyond the source_refs string.

13 SOLUTION A PROCESSING



14 SOLUTION A SOURCE PATH



CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

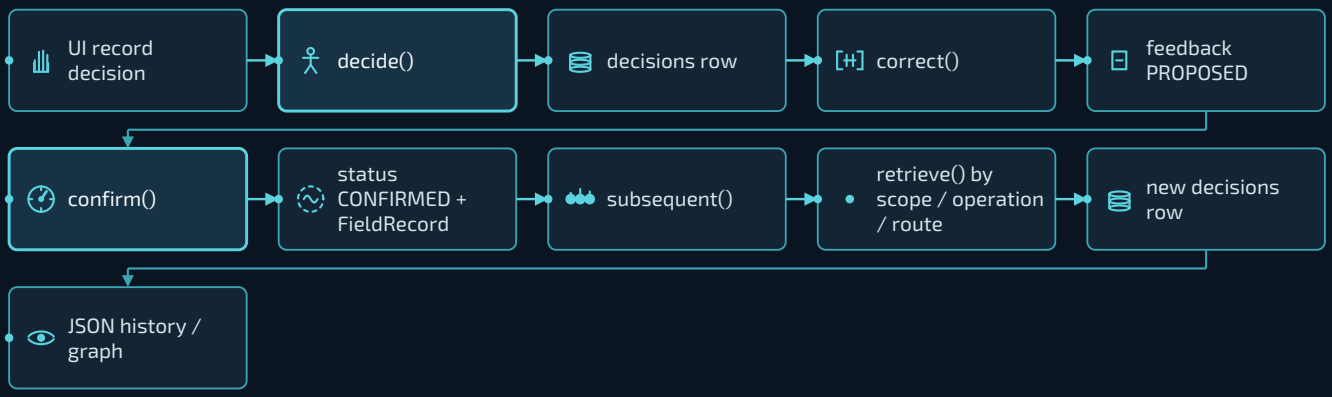
08 PROCESSING PATHS & INVERSE TRACES | B

§8 · RECURSIVE FEEDBACK & DECISION TRACE

FORWARD PROCESSING

11 PROCESSING STEPS

PROCESSING STEP = CALL OR DATA STEP IN THE REQUEST PATH · NOT A FIELDRECORD STATE TRANSITION (STATE MACHINE: P. 06)

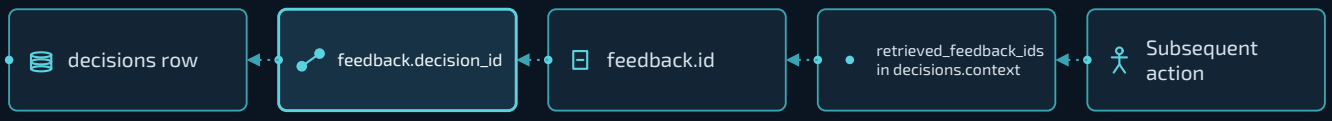


STEP 6 · CONFIRM(): FEEDBACK FIELD ENRICHED › CONFIRMED = FIELDRECORD STATE TRANSITION (P. 06) · FEEDBACK.STATUS STORED SEPARATELY

INVERSE TRACE

READ RIGHT TO LEFT · RESULT › SOURCE

INVERSE ARROWS = TRACE STEPS, NOT STORED RELATIONS

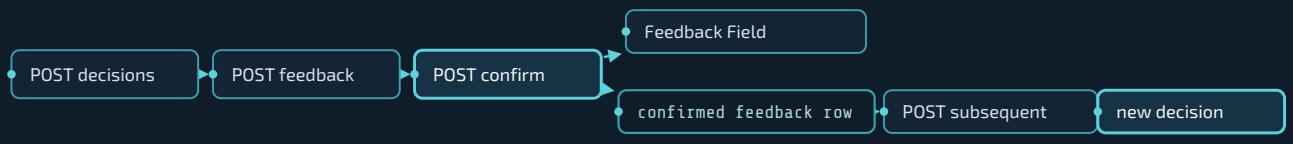


RETRIEVED_FEEDBACK_IDS = JSON REFERENCE IN DECISIONS.CONTEXT · LOGICAL ONLY · FIELDRELATION MISSING

PROVENANCE BOUNDARY

The FieldRelation table does not represent this path.

15 SOLUTION B FEEDBACK SEQUENCE



16 SOLUTION B RETRIEVAL FILTER



RETRIEVAL BOUNDARY

Filter has no shipment ID, carrier constraints or temporal expiry (see §3 B). No supersession or expiry logic (R7).

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

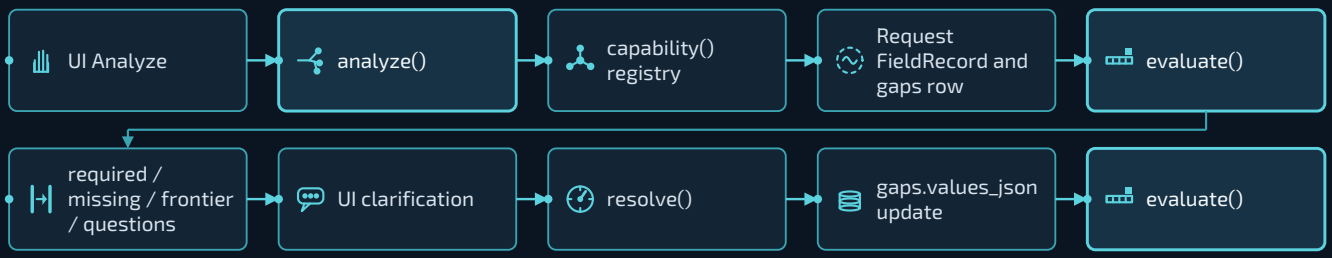
08 PROCESSING PATHS & INVERSE TRACES | C

§8 · UNDERSTANDING GAP DETECTOR

FORWARD PROCESSING

10 PROCESSING STEPS

PROCESSING STEP = CALL OR DATA STEP IN THE REQUEST PATH · NOT A FIELDRECORD STATE TRANSITION (STATE MACHINE: P. 06)

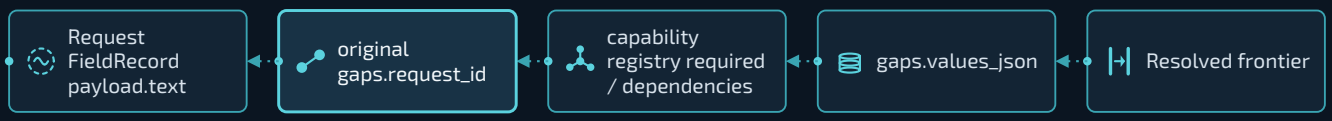


RE-EVALUATION CLOSES THE CLARIFICATION LOOP · RESOLVE() › EVALUATE()

INVERSE TRACE

READ RIGHT TO LEFT · RESULT › SOURCE

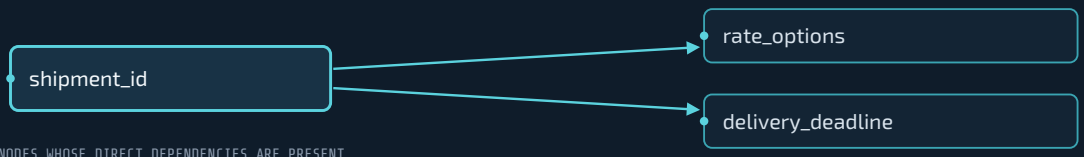
INVERSE ARROWS = TRACE STEPS, NOT STORED RELATIONS



PROVENANCE BOUNDARY

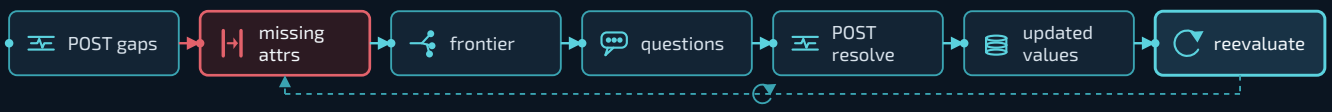
No relation edge is persisted.

17 SOLUTION C DEPENDENCY GRAPH



FRONTIER = NODES WHOSE DIRECT DEPENDENCIES ARE PRESENT

18 SOLUTION C CLARIFICATION SEQUENCE



SOLVER BOUNDARY

No graph cycle detection and no minimal global clarification set. Unknown capability defaults to booking.

09 FRONTEND ARCHITECTURE

§9 · DIAGRAM 10

◆ frontend/src/main.tsx owns the full App state, conditionally renders three tab views and calls the API client.

APP STATE · MAIN.TSX



API CLIENT · SERVICES/API.TS



GRAPH VIEW · COMPONENTS/FIELDGRAPH.TSX



CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

The UI shows raw JSON processing results, plus the graph when graph.fields is nonempty.

10 FRONTEND COMPONENTS



FRONTEND GAPS

- No dedicated state timeline visualization
- No persistence of UI state between reloads
- No actual dependency-graph React Flow for Solution C — the shared FieldGraph only uses persisted field relations
- No frontend test files
- No typed API models
- Reset is global across scopes

10 EXECUTION, CONFIGURATION & TEST ARCHITECTURE

§10 · DIAGRAMS 19 · 23 · 24

23 LOCAL STARTUP · CD NEMOX-DEVELOPER-TOOLS · ./START.SH

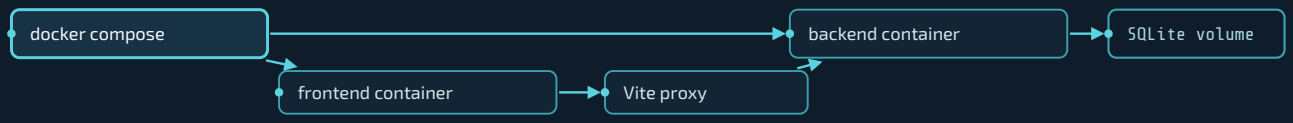


Requires Python3, npm and network access to install pip / npm packages unless already installed. Creates .venv, installs backend/requirements.txt, installs frontend dependencies if node_modules is missing, initialises SQLite, starts uvicorn at 127.0.0.1:8000 and Vite at 0.0.0.0:3000. Browser: http://localhost:3000.

19 RUNTIME DEPLOYMENT



24 DOCKER STARTUP · DOCKER COMPOSE UP --BUILD



DOCKER ELEMENT	CONFIGURATION
Backend	python:3.12-slim at 8000
Frontend	node:22-alpine at 3000
Vite proxy	DOCKER_ENV=1 targets backend:8000; local proxy targets 127.0.0.1:8000
Data	SQLite bind mount ./data:/app/data
Verification	Docker availability not verified in this environment

TEST ARCHITECTURE

PYTHONPATH=BACKEND PYTHON -M PYTEST -Q BACKEND/TESTS/TEST_SYSTEM.PY

CHECK	EVIDENCE
Backend tests	5 passed on 2026-11-01 — intent trace; feedback confirmation and route-scoped reuse; gap clarification; transition validation; relation scope mismatch
Frontend build	cd frontend && npm install && npm run build — not executed here, frontend dependencies absent
Browser E2E	not executed
Docker startup	not executed

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

11 RECURSIVE SEVEN-PASS GAP REGISTER

§11 · R1 – R7

◆ Seven recursive passes from domain to longitudinal context. Each pass separates the observed implemented path from its gap or extension.

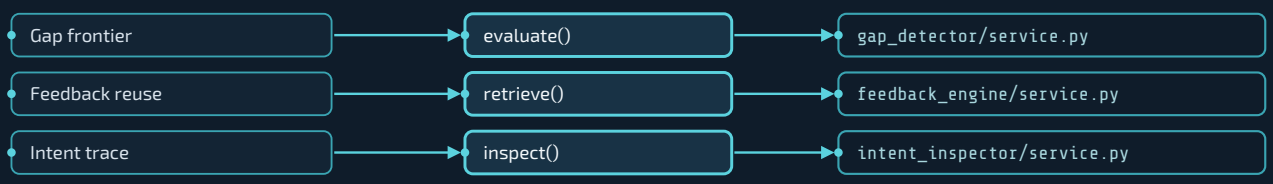
R1	Domain	Three deterministic logistics demos	No production freight integration	EXTENSION
R2	Fields	FieldRecord and 6 SQLite tables	F01 / F11 / F12 conceptual or derived	EXTENSION
R3	Components	API › service › sqlite3	No repository interfaces for all tables	EXTENSION
R4	Data	Request and correction persistence	C gaps not linked through FieldRelation	GAP
R5	State	Field state transition table	Feedback status separate; non-atomic confirmation	GAP
R6	Feedback	Confirmed correction matching	Operation + route + scope only; no validity dates	GAP
R7	Longitudinal	Confirmed feedback reused	No supersession or expiry logic	EXTENSION

12 ARCHITECTURE-CODE TRACEABILITY

§12 · REQUIREMENT › FIELD / FUNCTION › TEST · DIAGRAM 20

REQUIREMENT / RULE	FIELD / SOURCE FUNCTION	TEST
Interpret relative delivery constraint	F02 / F04 / F05; inspect() backend/intent_inspector/service.py	test_intent_trace
Historical context candidates	F03; HISTORY and inspect()	test_intent_trace
Relation source / target scope	F08; FieldRepository.relation()	test_relation_scope
Transition guard and version	F09; FieldRepository.transition()	test_state_validation
Feedback decision correlation	F06 / F07; correct()	test_feedback_scope_and_confirm
Confirm and reuse correction	F07 / F12; confirm(), retrieve(), subsequent()	test_feedback_scope_and_confirm
Gap dependency frontier	F10; capability(), evaluate()	test_gap_resolution
Clarification update	F10; resolve()	test_gap_resolution
React Flow visualization	F11; FieldGraph.tsx	Not covered by tests
One-command startup	start.sh; docker-compose.yml	Not end-to-end tested

20 REQUIREMENT TO CODE



CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

13 RENDERED ARCHITECTURE DIAGRAMS | INDEX

§13 · ALL 24 REDRAWN IN SYNTX FORM

◆ All 24 diagrams of the source are redrawn in this document at the point where their architecture is described. The index maps each one to its page.

01	System context	p.01	09	Backend components	p.07	17	Solution C dependency graph	p.11
02	Domain boundaries	p.03	10	Frontend components	p.12	18	Solution C clarification sequence	p.11
03	Shared field core	p.07	11	Data relationships	p.06	19	Runtime deployment	p.12
04	Twelve-field classification	p.04	12	API interaction	p.08	20	Requirement to code	p.13
05	Field lifecycle	p.06	13	Solution A processing	p.09	21	State and event storage	p.07
06	Field coupling A	p.05	14	Solution A source path	p.09	22	Context graph traversal	p.07
07	Field coupling B	p.05	15	Solution B feedback sequence	p.10	23	Local startup	p.12
08	Field coupling C	p.05	16	Solution B retrieval filter	p.10	24	Docker startup	p.12

14 REMAINING IMPLEMENTATION GAPS & DELIVERY ACCEPTANCE

§14

◆ The three status tiers are intentionally preserved for engineering handover: what runs, what is only derived, and what is concept.

● Implemented

- Three deterministic backend service workflows
- Field storage / relations / transitions
- Graph endpoint
- React tabs
- Docker configuration

◐ Derived only

- Longitudinal context retrieval
- Inspection projections

◌ Conceptual

- Twelve-field canonical object model
- Cross-workflow manifests
- Complete event sourcing
- Advanced dependency solver
- AI extraction
- Production-grade tenant boundaries

TEST EVIDENCE

5 / 5 backend tests passed · PYTHONPATH=backend → frontend build · not claimed → Docker startup · not claimed → browser E2E · not claimed

◆ DELIVERY BOUNDARY

This architecture documents the current source, not a claimed production-complete system.

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

LEGAL NOTICE | CONFIDENTIALITY | RESERVATION OF RIGHTS

BINDING FOR EVERY RECIPIENT, READER OR HOLDER OF THIS DOCUMENT

1 Ownership

This document — including its structure, SYNTX representation, architecture reconstruction, recursive and inverse sweep results, diagrams, status classification and all derived descriptions — is the confidential intellectual work of Ottavio Braun, SYNTX SYSTEM. All rights are reserved.

2 No licence

No right of use of any kind is granted. Receipt, possession, access to or knowledge of this document does not constitute a licence, transfer of rights, offer, order, contract, partnership or commitment of any kind.

3 Written approval required

Any use, reproduction, storage beyond inspection, distribution, disclosure to third parties, adaptation, translation, implementation, re-engineering, integration into products or services, and any use for training, evaluating or prompting AI systems — in whole or in part, directly or indirectly — requires the prior, explicit, written approval of Ottavio Braun. Approval is valid only if confirmed by Ottavio Braun personally, in text form, for a named purpose and recipient. Silence, conduct, payment or prior collaboration do not constitute approval.

4 Purpose limitation

Until such approval is given, this document may be read solely to evaluate whether a written agreement with Ottavio Braun is to be concluded. Any other purpose is excluded.

5 Return and deletion

On request, and in any case if no written approval is granted, all copies — physical and digital, including excerpts, notes and derived material — must be deleted or returned without delay and their deletion confirmed in writing.

6 No warranty

The document describes an inspected source-code state as of 1 November 2026. It makes no representation of completeness, correctness, fitness for a particular purpose, security or production readiness. Implemented, derived, conceptual and proposed elements are distinguished; none of them is warranted.

7 No liability

No liability is accepted for decisions, implementations or damages arising from use of this document without the written approval described in section 3.

8 Third-party names

nemox, FastAPI, React, Vite, SQLite, Docker, React Flow / @xyflow and other product names remain the property of their respective owners and are used solely to describe the inspected implementation.

9 Proprietary method

The SYNTX Recursive Deep Sweep and Inverse Sweep are proprietary procedures of Ottavio Braun. Their internal material is not reproduced and no right in them is conveyed by this document.

Deutsch · verbindlicher Hinweis

Vertraulich. Jede Nutzung, Vervielfältigung, Weitergabe, Bearbeitung oder Umsetzung dieses Dokuments — ganz oder teilweise — bedarf der vorherigen, ausdrücklichen, schriftlichen Freigabe durch Ottavio Braun. Ohne diese Freigabe wird kein Nutzungsrecht eingeräumt.

APPROVAL CONTACT

Ottavio Braun | SYNTX SYSTEM

mirror@syntx-system.com

MUNICH, NOVEMBER 2026

© 2026 OTTAVIO BRAUN · ALL RIGHTS RESERVED